

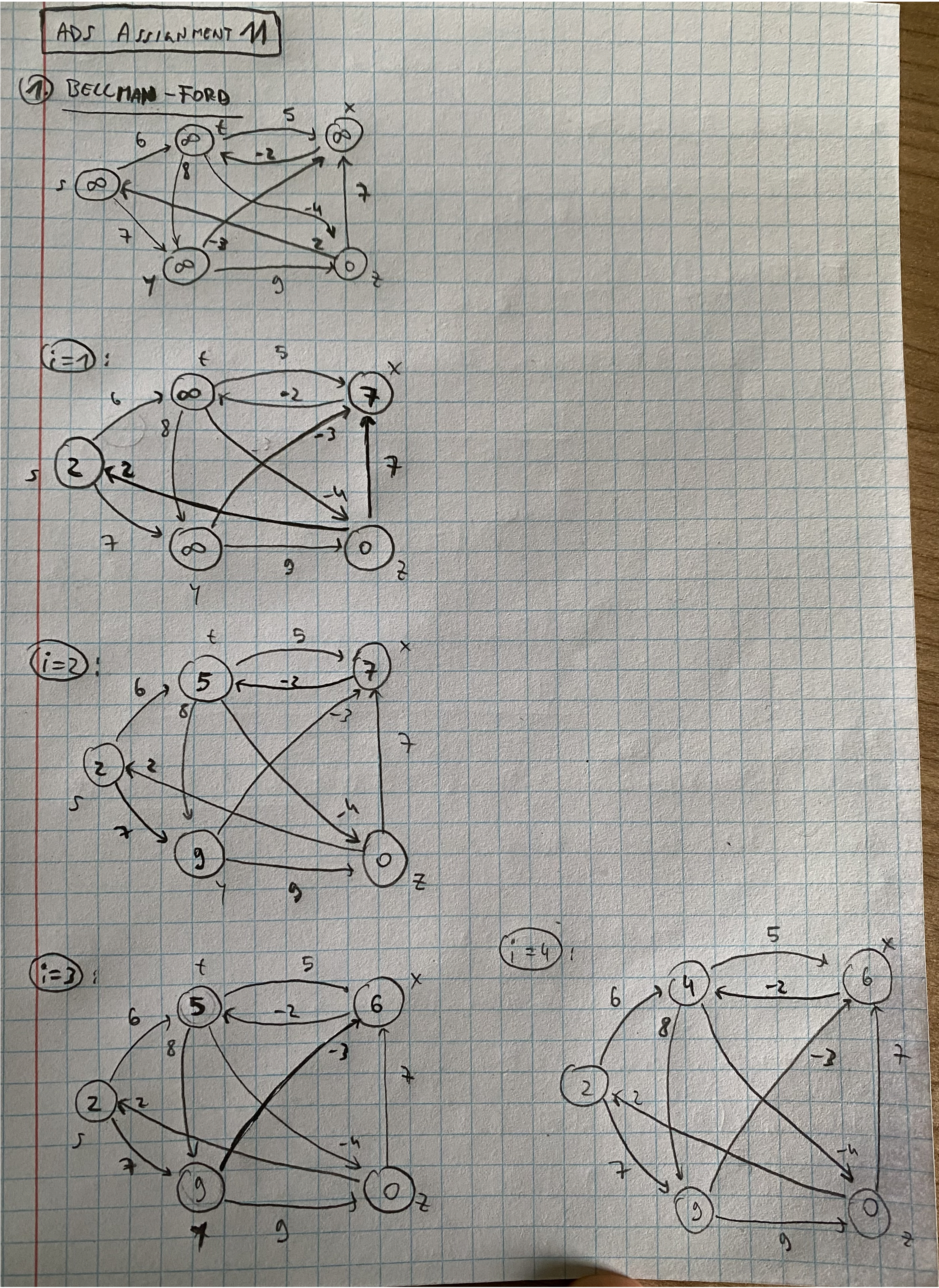
Assignment 11

Author: Lan Stare

Date: 28. 11. 2025

Question 1

Remark: I sincerely apologize to the person reading and judging my handwritten notes, I tried to make them better this week.



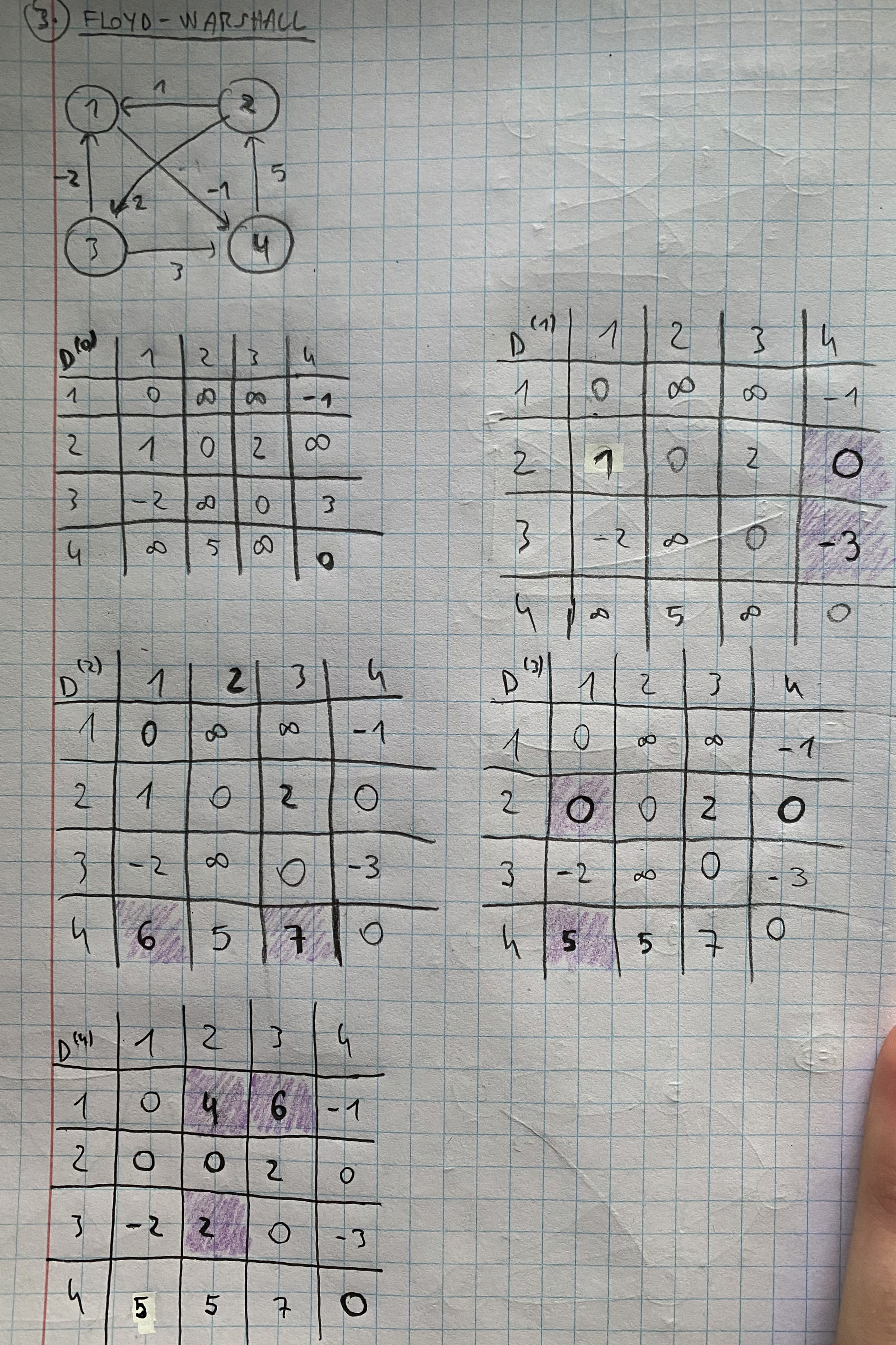
When we run the algorithm for $i = 5$, nothing changes so our shortest distances are stable.

Question 2

Since we have the starting and ending vertex given, we'll modify the Bellman-Ford algorithm, since it's faster for that particular case.

In the lecture we mentioned that we can optimize the algorithm to be of complexity $O(|V||E|)$ by iterating through edges for each $i \in 1, \dots, n$. Since our shortest path between any two edges has at most k -edges, our algorithm will stabilize after at most k -iterations. So if we know that our graph doesn't have a negative cycle, we don't need to change the algorithm to have complexity $O(k|E|)$ otherwise we add a stopping criterion that terminates after the k -th iteration.

Question 3



Question 4

If there is a negative cycle present in our graph, we can easily detect it with Floyd-Warshall's algorithm. If there is a negative cycle, there exists a vertex i , such that $D_{(i,i)}^n < 0$ for n vertices. This holds because in the n -th step we are free to use any vertex to get to any other vertex so if for example vertex u is an element of the negative cycle we can traverse the cycle to get back to u , which gives a negative weight (by definition of a negative cycle).

A little further thinking: we can even be sure that if in D^{n-1} we have no vertex i such that $D_{(i,i)}^{n-1} < 0$ holds, we can only check n -th element in the next step since if a neg cycle showed itself it definitely included vertex n .

Question 5

EXPLANATION:

If I understand the task correctly then we need to build a matrix that keeps track of the current predecessors (where we're only using vertices 1 to k at step k).

To do this we just need to also remember which predecessor changed the entry in the matrix. My implementation will not keep 2 separate matrices but will include both informations about shortest path and the predecessor in space the $[i][j]$ of each matrix.

```
In [4]: #implementation:
import numpy as np

def print_paths(matrix):
    """ prints the shortest paths between all pairs of vertices (one path per line) """
    n = len(matrix)
    D = []

    for k in range(n + 1):
        #|V| x |V| matrix (assuming its a list of rows)
        D[k] = []
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                if k == 0:
                    #our base case
                    D[k][i][j] = [matrix[i][j], i] if matrix[i][j] != np.inf else [np.inf, 0]
                    new_possible_w = D[k - 1][i][k][0] + D[k - 1][k][j][0]
                    old_possible_w = D[k - 1][i][j][0]
                    if new_possible_w < old_possible_w:
                        D[k][i][j] = [new_possible_w, D[k - 1][k][j][1]]
                    else:
                        D[k][i][j] = [D[k - 1][i][j][0], D[k - 1][k][j][1]]

    #now we need to print first all the rows (while checking that there is a path):
    for i in range(1, n + 1):
        list_of_vertices_for_i = []
        for j in range(1, n + 1):
            predecessor = D[n][i][j][1]
            if predecessor == 0:
                raise ValueError("There are no paths from {i} to {j} :3.")
            list_of_vertices_for_i.append(predecessor)
            #since we were building it backwards it needs to be reversed
            print(list_of_vertices_for_i)
            continue
        else:
            list_of_vertices_for_i.append(D[n][i][predecessor])
```